MUHAMMAD HASAN 02-135251-040

Data Structures and Algorithms

Assignment 02

Code 1:

```
main(){
    int sum=0;
    for(int i=0;i<n; i++){                    0 → n
        if(i>6)                                  } O(1)
            sum = sum+1;
        else if(i==0){
            for(int k=0; k<n; k++){           0 → n
                int y=n;10
                while(y>0){                   10 → 0      O(n)
                    sum = sum-1;                          } O(n²)
                    y--;
                }
            }
        }
        else if(i==2){
            int i=1;
            while(i<n){                       n → n/2^0
                int j=0;10
                while(j>0){                   n/2 → n/2^1
                    j = j/2;                  n/4 → n/2^2
                    i = 2*i;
                }                             n/8 → n/2^k
            }
        else{                                 O(log n)
            for(int p=0; p<n; p++){           0 → n
                for(int k=1; k<n; k*=2){      O(log n)
                    cout<<"*";
}}}
```

this will run only one time, because due to inner loop i will be > n.

i	j	
1	10	
2	5	
4	2	
8	1	
16	0	

simplified form of code 1

```
for (int i=0; i<n; i++){                     0 → n
    for (int k=0; k<n; k++){                 0 → n^2
        int y=n;
        while (y>0){
            y--;
        }
    }
}

int i=1;
while (i<n){                                 log n
    int j=n;
    while (j>0){
        j /=2;
        i *=2;
    }
}

for (int p=0; p<n; p++){
    for (int k=1; k<n; k*=2){                n log n
    }
}
}
```

$O(n) + O(n^2) + O(\log n) + O(n \log n)$

$\rightarrow O(n^2)$

Code 2:

```
void fun(int n){
    int i, count = 0;
    if(n > 4){
        for (i=1; i*i <= n; i++)
            count++;
    }
    else{
        int point = 0, i = 1;
        while(i < n){
            point++;
            i *= 2;
        }
    }
    for(int y=1; y<point; y*=2)
        cout << "We are playing";
}
```

$\sqrt{n}$

$\log n$

$\log(\log n)$

i	point
1 →	1
2 →	2
4 →	3
8 →	4

$\log(n)$

$n = 10$

i	i*i	worst
1	1 <= 10	↓
2	4 <= 10	$i^2 = n$
3	9 <= 10	$i \to \sqrt{n}$
4	16 <= 10	

$O(\sqrt{n}) + O(\log n) + O(\log(\log n))$

$\Rightarrow O(\sqrt{n})$

Code 3

```
void fun(int n){
    int i, j, k, count = 0;
    for(i = n/2; i <= n; i++){
        for(j = 1; j + n/2 <= n; j++){
            for(k = 1; k <= n; k *= 2)
                count++;
        }
    }
}
```

$n = 10$

i	j	j + n/2 <= n
5	1	6 <= 10
	2	7 <= 10
	3	8 <= 10
	4	9 <= 10
	5	10 <= 10

$i = n/2 \to n$: $n/2$

$j + \frac{n}{2} <= n$

$\to j <= n - \frac{n}{2} \to j <= \frac{n}{2}$

$\log n$

Since they all are nested loops

$O(\frac{n}{2}) \times O(\frac{n}{2}) \times O(\log n) \rightarrow O(n^2 \log n)$

Code 4:

```
void ful(int n){
    int i, j, k, count=0;
    cin >> i;
    if(i > 10){
        for(i = n/2; i<=n; i++){          O(n/2)
            for(j=1; j<=n; j=j+1){        O(n)
                for(k=1; k<=n; k*=2)      O(log n)
                    count++;
    }  }
}else{
    for(int y=n; y>0; y--){               O(n)
        cin >> k;
        if(k>0){
            int avg=90;
            avg+=20;
        }
}  }
```

$O(\frac{n}{2}) \times O(n) \times O(\log n) + O(n)$

$O(n^2 \log n) + O(n)$

$O(n^2 \log n)$

Code 5

```
void fun(int n){
    int i, j, k, count=0;
    for(i=n/2; i<=n; i++){                O(n/2)
        for(j=1; j<=n; j=j+1){            O(n)
            for(k=1; k<=n; k*=2)          O(log n)
                count++
    }
    int c=0;
    for(i=n; i>0; i/=2){                  O(log n)
        c++;
    }
log(log n) for(i=1; i<c; i*=2){
        cout<<"Hello";
    }
}
```

the c we'll get from upper loop will be of log n, because loop is running with the complexity

$$[O(n) \times O(n) \times O(\log n)] + O(\log n) + O(\log(\log n))$$

$$O(n^2 \log n)$$

Code 6:

```
main() {
    int s=0;
    for(int i=0; i<n; i++) {          0 → n-1
        if(i>j) sum+=1;               contributing if 1
        else {
            for(int k=0; k<n; k++){   0 → n-1
                int y=n;
                while(y>0){           n → 1
                    sum-=1;
                    y--;
                }
            }
        }
    }
    int i=1;
    while(i<n){                       log n
        int j=n;
        while(j>0)                    log n
            j/=2;
        i*=2;
    }
}
```

So, these all three are nested loops, as else block will execute most of the time

$\{O(n) \times O(n) \times O(n)\} + \{O(\log n) \times O(\log n)\}$

$O(n^3)$

Code 7:

```
void fun(){
    cout<<"DSA Course";

    if(x==1){
        for(int i=1; i<=n; i++){          1 → n
            for(int j=1; j<=n; j++){
                cout<<"*";                 only one time
                break;
            }
        }
    }else if(i==10){
        for(int p=0; p<n; p+=10){
            for(int y=0; y<n; y+=2)
                count--;
        }
    }
}
```

y	n=50
0	<10
2	<10
4	<10
6	<10
8	<10
↓	

n/2

p	n=50
0	0<50
10	10<50
20	20<50
30	30<50
40	40<50
50	50<50

n/2

Time Complexity:

$O(n) + \{O(\frac{n}{2}) \times O(\frac{n}{10})\}$

$O(n^2)$

Code 8:

```
void fn(int n){
    for(int i=0; i<n; i++){          0 → n-1
        for(int j=1; j<i*i; j++)     1 → i²
    }
}
```

```
{   i=0 ,   3 ×
    i=1 ,     ×
      2       1, 2, 3
      3       1, 2, 3, 4, 5, 6, 7, 8
```

For inner loop:

$i = 0, 1, 2, 3, \ldots, n-1$

$0^2, 1^2, 2^2, \ldots (n-1)^2$

$O(n^2)$

$\hookrightarrow O(n) \times O(n^2)$

$\hookrightarrow O(n^3)$

Date:

Question 2: $T(n) = T(n-1) + n$

expanding the recurrence

$\rightarrow T(n) = T(n-2) + (n-1) + n$

$\rightarrow T(n) = T(n-3) + (n-2) + (n-1) + n$

after k expansions, (we'll hit base case)

$\rightarrow T(n) = T(n-k) + (n-k+1) + (n-k+2) + \ldots + n$

let's suppose $n-k = 1$ & 1 is the base case

$n-k = 1 \rightarrow k = n-1$

Substituting

$T(n) = T(n-n+1) + (n-n+1+1) + (n-n+1+2) + \ldots + n$

$\rightarrow T(1) + 2 + 3 + \ldots + n$

$\rightarrow 2 + 3 + \ldots + n$

$\rightarrow \frac{n(n+1)}{2} - n$

So,

$T(n) = T(1) + \frac{n(n+1)}{2} - 1$

$\rightarrow O\left(\frac{n^2}{2}\right)$

$\rightarrow O(n^2)$ Ans